

Học Sâu (Deep Learning)

Chương 2: Các khối xây dựng toán học của Mạng Nơ-ron

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Ví dụ đầu tiên về Mạng Nơ-ron (MNIST)
2. Biểu diễn dữ liệu bằng Tensor
3. Phép toán Tensor (Tensor Operations)
4. Tối ưu hóa Dựa trên Gradient
5. Đồ thị Tính toán & Lan truyền ngược
6. Tổng kết

Ví dụ kinh điển: Tập dữ liệu MNIST

- **Bài toán:** Phân loại ảnh các chữ số viết tay (từ 0 đến 9).
- **Dữ liệu:** 60.000 ảnh huấn luyện, 10.000 ảnh kiểm tra.
- Kích thước mỗi ảnh: 28×28 pixels (ảnh xám đen trắng).
- Trong học máy:
 - Phân loại (Category) gọi là **Lớp (Class)**.
 - Dữ liệu đầu vào gọi là **Mẫu (Sample)**.
 - Kết quả của mẫu gọi là **Nhãn (Label)**.



Hình 2.1: Một số mẫu ảnh từ MNIST

Tải dữ liệu MNIST trong Keras

- Tập dữ liệu MNIST được tải sẵn trong Keras, dưới dạng 4 mảng NumPy.

```
1 from keras.datasets import mnist
2 (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
3
```

- **Tập huấn luyện (Training set):** train_images và train_labels
- **Tập kiểm tra (Test set):** test_images và test_labels
- Hình ảnh được mã hóa dưới dạng mảng NumPy, nhãn là các số từ 0 đến 9.

Khám phá hình dạng dữ liệu

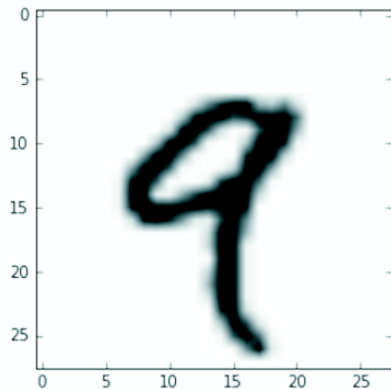
- Mảng NumPy là thư viện phổ biến để tính toán số. Nó được sử dụng làm định dạng lưu trữ trao đổi dữ liệu.
- Hãy xem xét cấu trúc của tập huấn luyện:

```
1 >>> train_images.shape
2 (60000, 28, 28)
3 >>> len(train_labels)
4 60000
5 >>> test_images.shape
6 (10000, 28, 28)
7
```

- Dữ liệu huấn luyện gồm 60.000 mẫu, mỗi mẫu là một ma trận 28×28 .

Xem xét một mẫu dữ liệu (Sample)

- Mỗi bức ảnh là một ma trận số nguyên.
- Giá trị từ 0 (trắng) đến 255 (đen).
- Ở đây chúng ta hiển thị mẫu thứ 4 trong tập huấn luyện (chữ số '9').



Hình 2.2: Biểu diễn ma trận của một ảnh

Kiến trúc của mô hình mạng nơ-ron

- Khối xây dựng cốt lõi của mạng nơ-ron là **lớp (layer)**.
- Các lớp trích xuất các biểu diễn từ dữ liệu được cung cấp.

```
1 import keras
2 from keras import layers
3
4 model = keras.Sequential(
5     [
6         layers.Dense(512, activation="relu"),
7         layers.Dense(10, activation="softmax"),
8     ]
9 )
10
```

- Lớp Dense là lớp được kết nối đầy đủ (fully connected).
- Lớp cuối cùng là softmax 10 chiều, trả về mảng 10 điểm xác suất.

Biên dịch (Compile) và Tiền xử lý dữ liệu

Biên dịch mô hình:

```
1 model.compile(  
2     optimizer="adam",  
3     loss="sparse_categorical_crossentropy",  
4     metrics=["accuracy"],  
5 )  
6
```

Tiền xử lý dữ liệu:

Biến đổi ảnh 28×28 thành vector 784 chiều, đưa giá trị về khoảng $[0, 1]$.

```
1 train_images = train_images.reshape  
    ((60000, 28 * 28))  
2 train_images = train_images.astype("float32") / 255  
3
```


Huấn luyện (Training) và Đánh giá (Evaluation)

- Quá trình huấn luyện (khớp dữ liệu) trong Keras được thực hiện thông qua `model.fit()`.

```
1 model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

- Sau khi huấn luyện, chúng ta đánh giá mô hình trên tập kiểm tra:

```
1 test_loss, test_acc = model.evaluate(test_images, test_labels)
2 print(f"test_acc: {test_acc}") # Khoảng 97.8%
```

- Hiện tượng mô hình hoạt động kém hơn trên dữ liệu mới so với dữ liệu huấn luyện gọi là **Overfitting** (Quá khớp).

Định nghĩa Tensor (Bộ căng)

Trong học sâu, mọi dữ liệu đều được biểu diễn thông qua các mảng đa chiều gọi là **Tensor**.

- **Vô hướng (Scalar) - Tensor 0D:** Chỉ chứa một con số duy nhất. Ví dụ: `np.array(12)`. Có 0 trục.
- **Vector - Tensor 1D:** Một mảng các con số. Trục duy nhất biểu diễn số chiều của vector.
- **Ma trận (Matrix) - Tensor 2D:** Một mảng của các vector (có Hàng và Cột).
- **Tensor 3D:** Đóng gói các ma trận thành hình hộp.
- **Tensor 4D, 5D:** Ứng dụng phổ biến cho chuỗi video.

Các thuộc tính chính của Tensor

Một tensor được xác định bởi ba thuộc tính cốt lõi:

- ➊ **Số trục (thứ hạng - rank):** Được biểu thị bằng `ndim` trong NumPy. Một ma trận có 2 trục, Tensor 3D có 3 trục.
- ➋ **Hình dạng (shape):** Là một tuple chứa số nguyên mô tả số chiều dọc theo mỗi trục. Ví dụ: (60000, 28, 28).
- ➌ **Kiểu dữ liệu (dtype):** Loại dữ liệu chứa trong tensor (float32, uint8, float64, bool v.v...).

Thao tác với Tensor: Cắt (Slicing) và Khái niệm Lô (Batch)

- **Cắt Tensor (Slicing):** Lựa chọn phần tử cụ thể dọc theo các trục.

```
1 my_slice = train_images[10:100] # Chọn từ index 10 đến 99
2 # Tương đương: train_images[10:100, :, :]
3 # Cắt ở giữa bức ảnh 14x14:
4 my_slice = train_images[:, 7:-7, 7:-7]
5
```

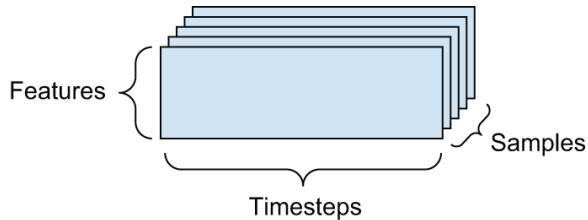
- **Khái niệm Lô (Batch):** Mô hình học sâu xử lý dữ liệu theo các đợt nhỏ. Trục đầu tiên (trục 0) thường được gọi là **trục lô (batch axis)**.

```
1 batch = train_images[:128] # Lô đầu tiên gồm 128 mẫu
2
```

Ví dụ thực tế: Vector và Dữ liệu Chuỗi thời gian

Dữ liệu Chuỗi thời gian (Tensor 3D)

- Hình dạng: (mẫu, thời gian, đặc trưng)
- Trục thời gian luôn là trục thứ hai (index 1).
- Ví dụ: Giá cổ phiếu mỗi phút (hiện tại, cao nhất, thấp nhất) trong 390 phút, 250 ngày giao dịch $\rightarrow$ Tensor (250, 390, 3).

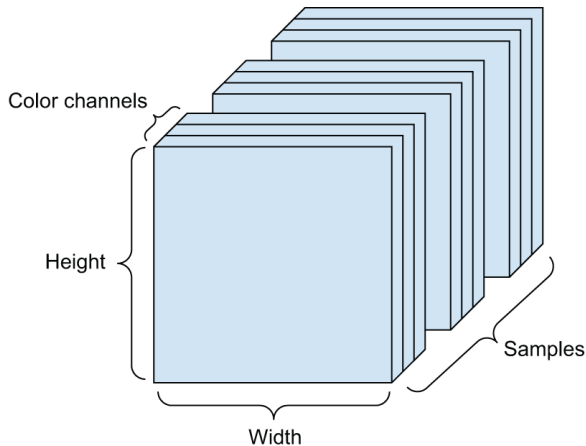


Hình 2.3: Tensor 3D cho dữ liệu chuỗi thời gian

Ví dụ thực tế: Dữ liệu Ảnh

Ảnh kỹ thuật số thường có 3 chiều: Chiều cao, Chiều rộng và Kênh màu.

- **Lô ảnh (Batch of images):** Tạo thành một **Tensor 4D**.
- Hình dạng: (mẫu, chiều cao, chiều rộng, kênh màu)
- Ví dụ: Lô 128 bức ảnh màu cỡ 256×256 có hình dạng là (128, 256, 256, 3).



Hình 2.4: Tensor 4D biểu diễn dữ liệu ảnh màu

Ví dụ thực tế: Dữ liệu Video

Dữ liệu video là trường hợp phức tạp nhất trong thực tế, sử dụng **Tensor 5D**.

- Một video là một chuỗi các khung hình (frames), mỗi frame là ảnh màu 3D.
- Hình dạng của tensor: (mẫu, khung hình, chiều cao, chiều rộng, kênh màu)
- *Ví dụ:* Một lô 4 clip YouTube 60 giây (144×256 pixels, 4 fps) có 240 khung hình sẽ có hình dạng (4, 240, 144, 256, 3).
- Kích thước này rất lớn (tương đương ~ 425 MB cho kiểu `float32`).

Phép toán Tensor: Element-wise và Broadcasting

Phép toán theo phần tử (Element-wise):

- Áp dụng độc lập cho từng mục trong tensor (Cộng, Trừ, Hàm ReLU...).
- Được thực thi song song qua BLAS siêu tốc trên kiến trúc GPU.

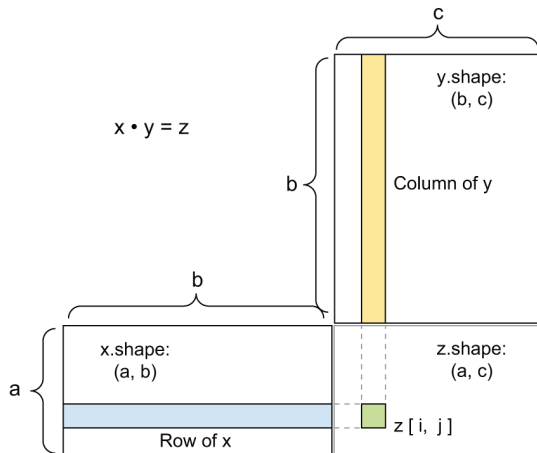
Phát sóng (Broadcasting):

- Cho phép tính toán giữa 2 tensor khác kích thước (VD: Ma trận + Vector).
- Tensor nhỏ được "lặp lại" dọc theo trục mới để khớp hình dạng.

```
1 import numpy as np
2 X = np.random.random((32, 10))
3 y = np.random.random((10,))
4 Z = X + y # y tu dong phat song thanh (32, 10) roi moi cong.
5
```


Sản phẩm Tensor (Tích vô hướng - Dot Product)

- Không giống phép nhân từng phần tử, **Tích Tensor** (`matmul` hoặc toán tử `@`) là phép toán phổ biến nhất.
- Tích ma trận:** Kết hợp các hàng ma trận 1 với cột ma trận 2.
- Khả năng tương thích hình dạng:
 $(a, b) \cdot (b, c) \rightarrow (a, c)$



Hình 2.5: Sơ đồ hộp minh họa tích ma trận

Định hình lại Tensor (Reshaping & Transposing)

- Định hình lại (Reshape) sắp xếp lại các hàng và cột để đạt được hình dạng mục tiêu mà không thay đổi tổng số phần tử.

```
1 x = np.array([[0., 1.],  
2             [2., 3.],  
3             [4., 5.]]) # Shape: (3, 2)  
4 x = x.reshape((6, 1)) # Tro thành cot 6 phan tu  
5
```

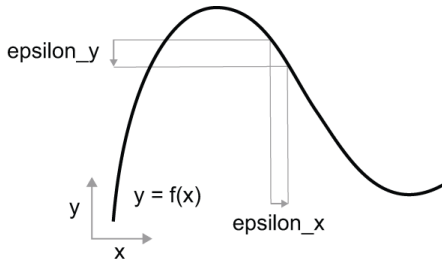
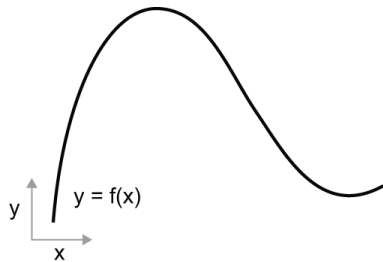
- Chuyển vị (Transpose): Hoán đổi hàng thành cột.

```
1 x = np.zeros((300, 20))  
2 x = np.transpose(x) # Shape tro thanh (20, 300)  
3
```

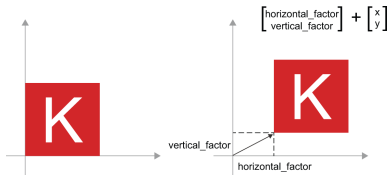
Ý nghĩa Hình học (1): Hàm số và Tính liên tục

Tất cả phép toán tensor đều có ý nghĩa biến đổi hình học.

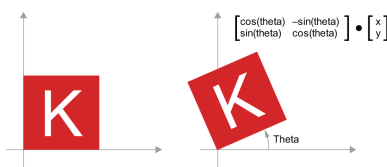
- Một mạng nơ-ron thực chất là một **hàm toán học** khổng lồ.
- Hàm số này phải **liên tục** và **trơn**.



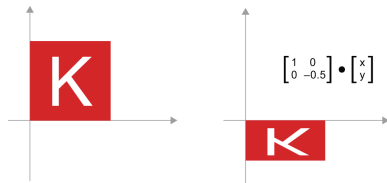
Ý nghĩa Hình học (2): Tịnh tiến, Phóng to, Quay



Hình 2.7: **Tịnh tiến:** Dời hình bằng phép cộng vector



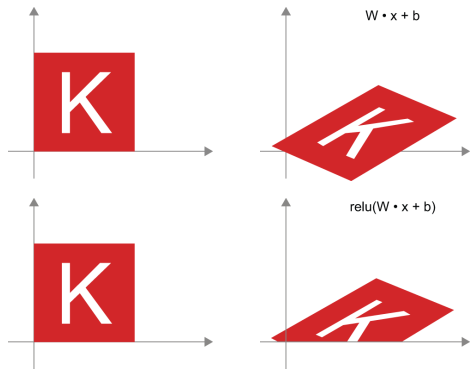
Hình 2.8: **Quay:** Tích ma trận 2×2 (Lượng giác)



Hình 2.9: **Thu phóng:** Tích ma trận đường chéo

Ý nghĩa Hình học (3): Phép biến đổi Affine & Dense layer

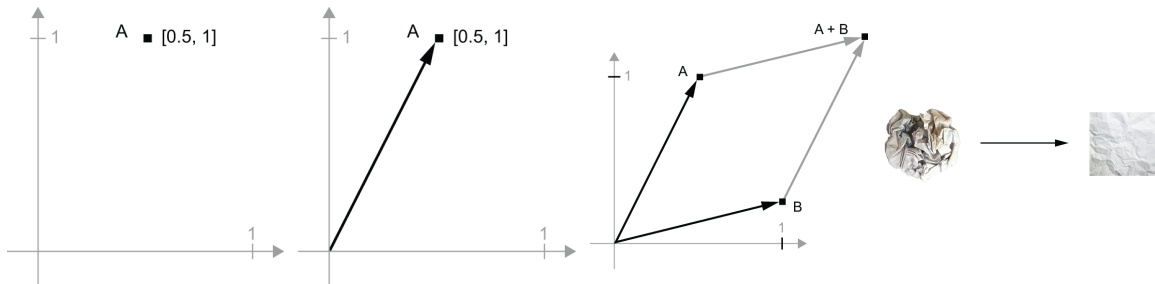
- **Phép biến đổi Affine** là sự kết hợp của biến đổi tuyến tính (nhân W) và tịnh tiến (cộng b).
- Lớp Dense: $y = W \cdot x + b$.
- **Kích hoạt phi tuyến (ReLU)**: Giúp bóp méo không gian affine, tạo ra các ranh giới phức tạp (để phân tách dữ liệu).



Hình 2.10: Biến đổi Affine & theo sau là ReLU

Ý nghĩa Hình học (4): Gỡ rối dữ liệu

Học sâu bản chất là chuỗi biến đổi hình học tinh vi để "gỡ rối" (untangle) các đa tạp dữ liệu phức tạp đan xen nhau.



Hình 2.11: Quá trình phân tách dữ liệu qua các lớp ẩn giống như việc từ từ vuốt phẳng một quả bóng giấy vò nát.

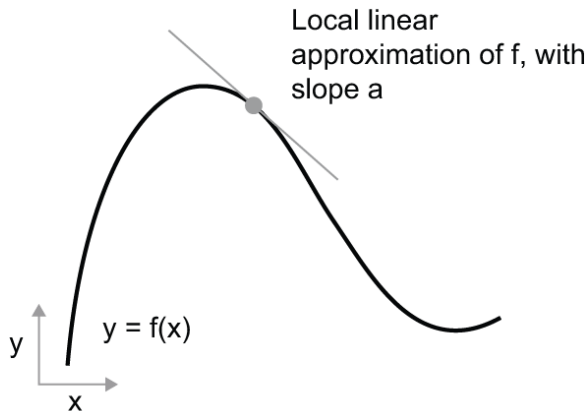
Vòng lặp Đào tạo (Training Loop)

- Khởi tạo ngẫu nhiên trọng số mô hình W và b .
- Lặp lại các bước sau (Vòng lặp đào tạo):
 - ① Lấy một lô ngẫu nhiên dữ liệu huấn luyện x và nhãn y_{true} .
 - ② **Forward Pass:** Chạy mô hình trên x để dự đoán y_{pred} .
 - ③ Tính hàm Mất mát (Loss) để đo lường sai số.
 - ④ Cập nhật tất cả các trọng số theo hướng giảm Loss.
- Làm sao để biết thay đổi trọng số bao nhiêu và hướng nào? Phương pháp: **Gradient Descent**.

Khái niệm Đạo hàm (Derivative)

Đạo hàm $f'(x)$ thể hiện **độ dốc (slope)** của tiếp tuyến với hàm số $f(x)$ tại điểm x .

- Nếu $f'(x) > 0$, tăng x sẽ làm hàm tăng.
- Nếu $f'(x) < 0$, tăng x sẽ làm hàm giảm.
- $\Rightarrow$ Muốn giảm giá trị $f(x)$ (Loss), chỉ cần đi **ngược hướng** với đạo hàm.

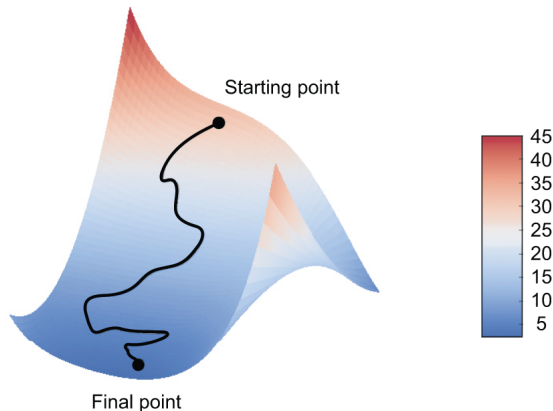


Hình 2.12: Đạo hàm là hệ số góc của tiếp tuyến

Đạo hàm của Tensor: Gradient

Sự tổng quát hóa của đạo hàm trong không gian đa chiều (nhiều tham số trọng số) gọi là **Gradient**.

- Gradient biểu thị **độ cong** của bề mặt hàm Loss đa chiều.
- Vectơ Gradient luôn chỉ về hướng hàm L tăng nhanh nhất.
- **Ngược hướng Gradient** là con đường ngắn nhất xuống thung lũng Loss.

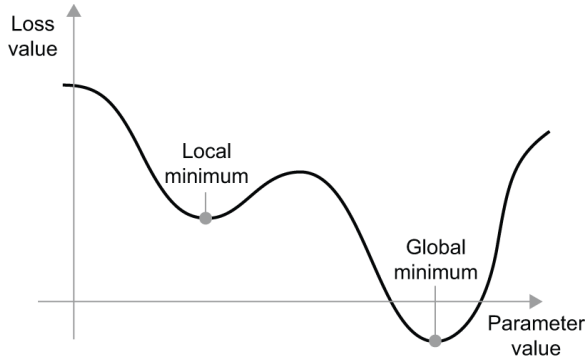


Hình 2.13: Mặt cong 3D của Loss function

Cực tiểu Toàn cục và Cực tiểu Cục bộ

Mục tiêu của học sâu là tìm **Cực tiểu toàn cục (Global Minimum)**, nơi sai số đạt mức thấp nhất.

- Tại cực tiểu, Gradient bằng 0 ($\nabla L = 0$).
- Với bề mặt Loss phức tạp, mô hình rất dễ mắc kẹt tại **Cực tiểu cục bộ (Local Minimum)**.



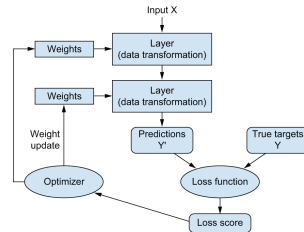
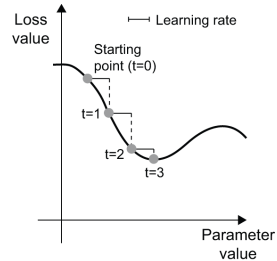
Hình 2.14: Local Minimum vs Global Minimum

Thuật toán SGD và Momentum

Stochastic Gradient Descent (SGD): Cập nhật dựa trên một mẻ nhỏ ngẫu nhiên.

$$W \leftarrow W - \text{learning_rate} \times \nabla L$$

- **Động lượng (Momentum):** Lấy cảm hứng từ một quả bóng lăn dốc, sử dụng vận tốc quá khứ để giúp quả bóng thoát khỏi các cực tiểu cục bộ hẹp.

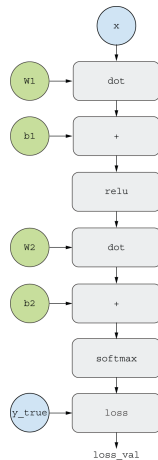


Hình 2.15: Học Gradient ngẫu nhiên & Vòng lặp

Đồ thị Tính toán (Computation Graph)

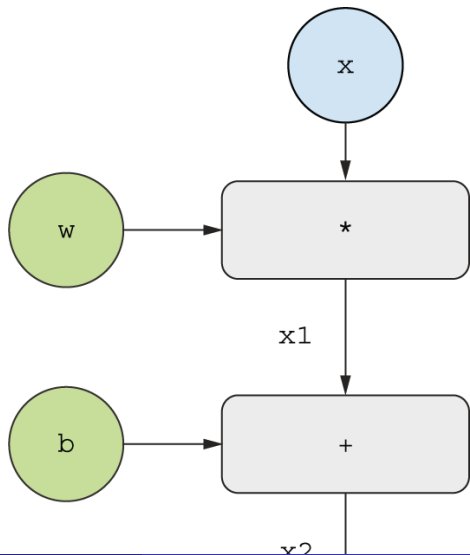
Làm sao để tính đạo hàm của hàng triệu trọng số? Sử dụng **Lan truyền ngược**.

- Cốt lõi của nó là **Đồ thị tính toán**.
- Các phép toán Tensor trở thành các "nút" (node), dữ liệu trở thành các cạnh.
- Biến tính toán toán học thành dữ liệu cấu trúc phục vụ máy tính.



Hình 2.16: Biểu diễn đồ thị cho mô hình mạng nơ-ron

Ví dụ lan truyền thuận (Forward Pass)

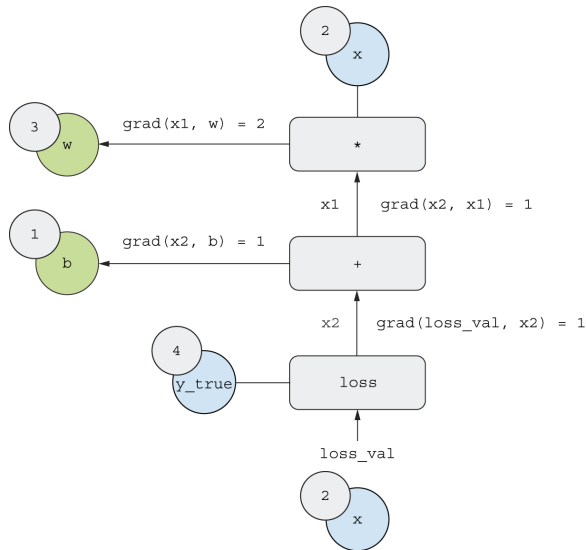


Lan truyền ngược (Backpropagation) & Quy tắc Chuỗi

Bằng cách áp dụng **Quy tắc chuỗi (Chain Rule)**:

$$\frac{d(f(g(x)))}{dx} = f'(g(x)) \cdot g'(x)$$

- Bắt đầu từ Loss, ta lan truyền đạo hàm



Tự động phân biệt với TensorFlow GradientTape

- Thực tế, Framework hiện đại như TensorFlow tự động hóa toàn bộ việc Lan truyền ngược bằng **Automatic Differentiation**.

```
1 import tensorflow as tf
2
3 x = tf.zeros(shape=()) # Gia tri khoi tao
4 with tf.GradientTape() as tape:
5     # Tape (Bang ghi) se luu lai do thi cac phep toan
6     y = 2 * x + 3
7
8 # Yeu cau Tape tinh gradient cua y doi voi x (dao ham)
9 grad_of_y_wrt_x = tape.gradient(y, x)
10
```

Thực hành (Từ đầu): NaiveDense

- Hãy xây dựng thủ công một lớp Dense sử dụng tensor ops.

```
1 import keras
2 from keras import ops
3
4 class NaiveDense:
5     def __init__(self, input_size, output_size, activation=None):
6         self.activation = activation
7         self.W = keras.Variable(shape=(input_size, output_size),
8                                   initializer="uniform")
9         self.b = keras.Variable(shape=(output_size,), initializer="zeros")
10
11     def __call__(self, inputs): # Thực hiện Forward Pass
12         x = ops.matmul(inputs, self.W) + self.b
13         if self.activation:
14             x = self.activation(x)
15         return x
16
```


Thực hành (Từ đầu): Vòng lặp Huấn luyện

- Khái quát lại quá trình `model.fit()` bằng đoạn code sau:

```
1 def one_training_step(model, images_batch, labels_batch):
2     with tf.GradientTape() as tape:
3         # 1. Forward pass để dự đoán và tính Loss
4         predictions = model(images_batch)
5         loss = ops.sparse_categorical_crossentropy(labels_batch, predictions)
6
7         average_loss = ops.mean(loss)
8
9         # 2. Backward pass (Tính gradient tự động nhờ Tape)
10        gradients = tape.gradient(average_loss, model.weights)
11
12        # 3. Cập nhật trọng số bằng Optimizer (như SGD)
13        update_weights(gradients, model.weights)
14
15    return average_loss
```

Tổng kết Chương 2

- **Tensors:** Nền tảng cấu trúc dữ liệu của học sâu. Bao gồm các bậc 0D, 1D, 2D, 3D, 4D v.v.
- **Phép toán Tensor:** Phép nhân ma trận, phép cộng, ReLU... là những biến đổi hình học tinh vi giúp gỡ rối đa diện dữ liệu.
- **Trọng số (Weights):** Tham số học hỏi chứa toàn bộ "kiến thức" của mô hình mạng.
- **Loss function & Optimizers:** Loss đo lường độ chính xác (mục tiêu cần tối thiểu hóa). Optimizer như SGD giảm Loss thông qua đi ngược Gradient.
- **Backpropagation:** Sử dụng Đồ thị tính toán và Quy tắc chuỗi để tính đạo hàm ngược một cách nhanh chóng qua cơ chế phân biệt tự động (GradientTape).